Rivanna Faithful 537 Sweep — For Dummies

This guide runs the **faithful 537 simulation sweep** on Rivanna. "Faithful" means it uses the current `537_Sports_Simulation.ipynb` mechanics only: no new congestion or opportunity penalty terms.

What The Rivanna Agent Needs To Know

If you open a fresh Cursor window on Rivanna, tell the agent:

```
Please run the faithful 537 Rivanna sweep.

Use this runbook:
sports/documents/Rivanna_Faithful_537_Sweep_For_Dummies.md

First do the preflight checks in that file.
Then submit Stage 1, Stage 2, and Merge with Slurm dependencies.
Track the jobs with scripts/track_slurm.sh and squeue.
Do not edit the model unless I ask.
```

The required files are:

```
sports/outputs/simulation_sweeps/faithful_537_sweep.py
sports/outputs/simulation_sweeps/faithful_537_sweep_rivanna_worker.py
sports/outputs/simulation_sweeps/rivanna_stage1_faithful_537.slurm
sports/outputs/simulation_sweeps/rivanna_stage2_array_faithful_537.slurm
sports/outputs/simulation_sweeps/rivanna_merge_faithful_537.slurm
scripts/track_slurm.sh
scripts/clear_slurm.sh
```

For This Sweep: The Simple Step-By-Step

Step 1: On The Mac, Push Only The Sweep Files

Do:

```
./scripts/rsync_push_to_hpc.sh sweep
```

Because: Rivanna needs the sweep Python scripts, Slurm scripts, and this guide before the jobs can run.

Be careful of: do **not** use `./scripts/rsync_push_to_hpc.sh all` for this sweep. That broad sync can touch unrelated project folders.

In case of trouble: run a dry run first:

```
DRY_RUN=1 ./scripts/rsync_push_to_hpc.sh sweep
```

The dry run should mention `sports/outputs/simulation_sweeps/`, not `python_packages/dblp-parser` or `tenure/tenure_pipeline`.

Step 2: Open Cursor On Rivanna And Go To The Repo

Do:

```
cd ~/Ivy_Net
```

Because: the Slurm scripts assume you submit from the repo root, where `sports/`, `scripts/`, and `pipe_job.slurm` live.

Be careful of: if your Rivanna repo is not at `~/Ivy_Net`, use the actual repo path.

In case of trouble: check where you are:

```
pwd
ls sports scripts pipe_job.slurm
```

Step 3: Run The Preflight Checks

Do:

```
ls sports/outputs/simulation_sweeps/faithful_537_sweep.py
ls sports/outputs/simulation_sweeps/faithful_537_sweep_rivanna_worker.py
ls sports/outputs/simulation_sweeps/rivanna_stage1_faithful_537.slurm
ls sports/outputs/simulation_sweeps/rivanna_stage2_array_faithful_537.slurm
ls sports/outputs/simulation_sweeps/rivanna_merge_faithful_537.slurm
ls scripts/track_slurm.sh
module load miniforge
~/.conda/envs/sports_net/bin/python --version
```

Because: this catches missing files or environment problems before submitting a multi-hour job chain.

Be careful of: if `sports_net` is missing, the scripts may fall back to another Python. That might work, but only if it has `numpy` and `pandas`.

In case of trouble: either fix the environment on Rivanna or submit with a different env name, for example:

```
ENV_NAME=my_env_name sbatch sports/outputs/simulation_sweeps/rivanna_stage1_faithful_537.slurm
```

Step 4: Submit Stage 1, Stage 2, And Merge In One Dependency Chain

Do:

```
j1=$(sbatch --parsable \
  sports/outputs/simulation_sweeps/rivanna_stage1_faithful_537.slurm)

j2=$(sbatch --parsable --dependency=afterok:$j1 \
  sports/outputs/simulation_sweeps/rivanna_stage2_array_faithful_537.slurm)

sbatch --dependency=afterok:$j2 \
  sports/outputs/simulation_sweeps/rivanna_merge_faithful_537.slurm

echo "Stage 1: $j1"
echo "Stage 2: $j2"
```

How to enter Step 4 (terminal mechanics):

1. Use a **bash** shell on Rivanna (the default login shell is usually fine).
2. `cd` to **Ivy_Net repo root** first (same as Step 2), so paths like `sports/outputs/...` exist.
3. Copy the **whole block** above and paste it once. Lines ending in `\` mean "continued on the next line"; the shell runs it as one multi-line command sequence. Press **Enter** after the last line (`echo "Stage 2: $j2"`).
4. What it does:
   - `sbatch --parsable` prints **only the numeric job ID** (no extra words), so it can be stored in `j1` / `j2`.
   - `--dependency=afterok:JOBID` tells Slurm: start this job only after that job **finishes with exit code 0**.
5. The **merge** job is submitted by the third line; Slurm prints its job ID on that line — note it if you want to `track_slurm.sh MERGE_JOBID`.

Same logic **without** line breaks (easier to type line-by-line):

```
j1=$(sbatch --parsable sports/outputs/simulation_sweeps/rivanna_stage1_faithful_537.slurm)
j2=$(sbatch --parsable --dependency=afterok:$j1 sports/outputs/simulation_sweeps/rivanna_stage2_array_fa
sbatch --dependency=afterok:$j2 sports/outputs/simulation_sweeps/rivanna_merge_faithful_537.slurm
echo "Stage 1: $j1"
echo "Stage 2: $j2"
```

Because: Stage 2 needs Stage 1 output, and Merge needs all Stage 2 shard outputs. The dependency chain prevents starting a later stage too early.

Be careful of: do not submit Stage 2 by itself unless `stage1_results.csv` already exists.

In case of trouble: check the queue:

```
squeue -u dzk3ja
```

Step 5: Track The Jobs

Do:

```
./scripts/track_slurm.sh
squeue -u dzk3ja
```

Because: `track_slurm.sh` tails the latest Slurm error log, while `squeue` shows whether the jobs are queued, running, or finished.

Be careful of: `Ctrl+C` only stops watching a log. It does not cancel the Slurm job.

In case of trouble: cancel a bad job only if needed:

```
scancel JOBID
```

Step 6: Check Results On Rivanna

Do:

```
ls -lh sports/outputs/simulation_sweeps/rivanna_faithful_537/
ls sports/outputs/simulation_sweeps/rivanna_faithful_537/stage2_shards/ | head
```

Because: the final results should land under `rivanna_faithful_537/`, with merged candidate files and plots.

Be careful of: if `stage2_shards/` is empty, Stage 2 probably has not started, failed early, or is still queued.

In case of trouble: inspect the relevant Slurm logs:

```
tail -f slurm-537_stage1-*.out
tail -f slurm-537_stage2-*_0.out
tail -f slurm-537_merge-*.out
```

Step 7: On The Mac, Pull Only The Sweep Results Back

Do:

```
./scripts/rsync_pull_from_hpc.sh sweep
```

Because: this brings back generated sweep results without using Git and without pulling source code back from Rivanna over your local edits.

Be careful of: do **not** use `./scripts/rsync_pull_from_hpc.sh all` for this sweep.

In case of trouble: dry run first:

```
DRY_RUN=1 ./scripts/rsync_pull_from_hpc.sh sweep
```

Step 8: Inspect The Candidate File

Do:

```
sports/outputs/simulation_sweeps/rivanna_faithful_537/grouped_candidates.csv
```

Because: this is the first file to inspect for stable right-side downturn candidates.

Be careful of: one lucky seed is not enough. Look for rows where the grouped/stability columns show the downturn repeats across seeds.

In case of trouble: open:

```
sports/outputs/simulation_sweeps/rivanna_faithful_537/README.md
```

That file should summarize what the merge step found.

Do I Run The Stage Files In Succession?

Yes. The stages depend on each other:

1. **Stage 1** runs a broad screen and writes `stage1_results.csv`.
2. **Stage 2** reads `stage1_results.csv`, then runs a Slurm array to verify candidate settings across seeds.
3. **Merge** reads all Stage 2 shard files and creates the final ranked candidate table and plots.

The easiest way is to submit them with Slurm dependencies, so each starts only after the prior stage finishes successfully.

Start In Cursor On Rivanna

Open Rivanna using `Rivanna.code-workspace`.

In the Rivanna terminal, go to the repo root, meaning the folder that contains `sports/`, `scripts/`, and `pipe_job.slurm`.

```
cd ~/Ivy_Net
```

If your repo is somewhere else on Rivanna, `cd` there instead.

Preflight Checks On Rivanna

Before submitting, run these from the repo root:

```
pwd
ls sports/outputs/simulation_sweeps/faithful_537_sweep.py
ls sports/outputs/simulation_sweeps/faithful_537_sweep_rivanna_worker.py
ls sports/outputs/simulation_sweeps/rivanna_stage1_faithful_537.slurm
ls sports/outputs/simulation_sweeps/rivanna_stage2_array_faithful_537.slurm
ls sports/outputs/simulation_sweeps/rivanna_merge_faithful_537.slurm
ls scripts/track_slurm.sh
```

Check that Rivanna can see Python in the `sports_net` environment:

```
module load miniforge
~/.conda/envs/sports_net/bin/python --version
```

If that path does not exist, find Python with:

```
which python
```

The Slurm scripts will fall back to `command -v python`, but the preferred environment is `sports_net`.

Optional quick import check:

```
~/.conda/envs/sports_net/bin/python - <<'PY'
import numpy, pandas
print("numpy", numpy.__version__)
print("pandas", pandas.__version__)
try:
    import matplotlib
    print("matplotlib", matplotlib.__version__)
except ModuleNotFoundError:
    print("matplotlib missing; numeric sweep can still run, plots may be skipped")
PY
```

If files are missing on Rivanna, go back to the Mac Cursor window and push only the sweep folder:

```
./scripts/rsync_push_to_hpc.sh sweep
```

Do **not** use `./scripts/rsync_push_to_hpc.sh all` for this sweep. The `all` shortcut is a broad project sync and includes unrelated folders such as `python_packages/dblp-parser`.

Submit The Whole Chain

From the repo root:

```
j1=$(sbatch --parsable \
  sports/outputs/simulation_sweeps/rivanna_stage1_faithful_537.slurm)

j2=$(sbatch --parsable --dependency=afterok:$j1 \
  sports/outputs/simulation_sweeps/rivanna_stage2_array_faithful_537.slurm)

sbatch --dependency=afterok:$j2 \
  sports/outputs/simulation_sweeps/rivanna_merge_faithful_537.slurm
```

That submits:

- Stage 1 as one job.
- Stage 2 as a 64-task array.
- Merge as one final job after all Stage 2 tasks succeed.

Print the job IDs so you can track them:

```
echo "Stage 1: $j1"
echo "Stage 2: $j2"
```

The final merge job ID is printed by the last `sbatch` command.

Monitor Progress

Use the existing tracker:

```
./scripts/track_slurm.sh
```

With no argument, it tails the most recent `slurm-*.err` file. With a job ID, it tails that job's `.err` file:

```
./scripts/track_slurm.sh JOBID
```

Or check the queue:

```
squeue -u dzk3ja
```

Or tail logs directly. Stage 1 and Merge write one log each:

```
tail -f slurm-537_stage1-*.out
tail -f slurm-537_merge-*.out
```

Stage 2 is an array job, so it writes one log per array task:

```
tail -f slurm-537_stage2-*_0.out
tail -f slurm-537_stage2-*_0.err
```

Press `Ctrl+C` to stop watching a log. That does **not** cancel the job.

To cancel a job:

```
scancel JOBID
```

To clean old Slurm logs before a new run:

```
./scripts/clear_slurm.sh
```

Use that only if you no longer need the old logs.

Where Results Land

Rivanna outputs land here:

```
sports/outputs/simulation_sweeps/rivanna_faithful_537/
```

Important files after completion:

```
stage1_results.csv
stage2_results_merged.csv
grouped_candidates.csv
candidate_plots/
README.md
```

The file to inspect first is:

```
sports/outputs/simulation_sweeps/rivanna_faithful_537/grouped_candidates.csv
```

Look for rows where:

```
moderate_stable = True
```

That means the setting produced a stable moderate right-side downturn under the rule we chose.

During the run, quick progress checks:

```
ls -lh sports/outputs/simulation_sweeps/rivanna_faithful_537/
ls sports/outputs/simulation_sweeps/rivanna_faithful_537/stage2_shards/ | head
```

Syncing Between Mac And Rivanna

The rsync shortcut for this job is:

```
sweep
```

Push code/scripts to Rivanna:

```
./scripts/rsync_push_to_hpc.sh sweep
```

Pull results back to your Mac:

```
./scripts/rsync_pull_from_hpc.sh sweep
```

Generated results are gitignored, but rsync can still move them.

Important safety rule:

- Use `./scripts/rsync_push_to_hpc.sh sweep` to send sweep code/scripts to Rivanna.
- Use `./scripts/rsync_pull_from_hpc.sh sweep` to bring generated sweep results back.
- Do **not** use broad `all` syncs for the sweep run.
- Use `DRY_RUN=1` before a sync when unsure:

```
DRY_RUN=1 ./scripts/rsync_push_to_hpc.sh sweep
DRY_RUN=1 ./scripts/rsync_pull_from_hpc.sh sweep
```

Rsync does not know Git history. The sweep shortcut is intentionally directional: push excludes generated results, and pull brings back generated results only so it does not overwrite sweep source code on the Mac.

If I Accidentally Ran A Broad Sync

If you accidentally ran:

```
./scripts/rsync_push_to_hpc.sh all
```

and it started pushing unrelated files, stop it with `Ctrl+C`.

For this sweep, `all` is not the right command. Use:

```
./scripts/rsync_push_to_hpc.sh sweep
```

The warning below is from SSH and is not the immediate problem:

```
WARNING: connection is not using a post-quantum key exchange algorithm.
```

The real problem is that `all` can sync unrelated project trees. We specifically do not want accidental DBLP XML pushes or broad code/data overwrites while preparing this sweep.

On Rivanna: Check For A Partial DBLP Transfer

If the accidental sync reached `dblp.xml` and you interrupted it, check Rivanna for partial temp files:

```
ls -lh ~/Ivy_Net/python_packages/dblp-parser/dblp.xml* \
  ~/Ivy_Net/python_packages/dblp-parser/.dblp.xml* \
  2>/dev/null
```

Normal `rsync` usually writes a hidden temporary file first and renames it only after a successful transfer. That means interrupting usually does **not** replace the existing remote `dblp.xml`, but it can leave a hidden partial temp file.

If you see a hidden temp file like `.dblp.xml.*`, remove only that temp file:

```
rm -f ~/Ivy_Net/python_packages/dblp-parser/.dblp.xml*
```

Do **not** delete the real `dblp.xml` unless you intentionally want to remove the full DBLP dump.

On The Mac: Use Dry Runs Before Syncing

Before pushing or pulling when unsure:

```
DRY_RUN=1 ./scripts/rsync_push_to_hpc.sh sweep
DRY_RUN=1 ./scripts/rsync_pull_from_hpc.sh sweep
```

The dry run should show only:

```
sports/outputs/simulation_sweeps/
```

If it shows `python_packages/dblp-parser`, `tenure/tenure_pipeline`, or another unrelated tree, stop and do not run the real command.

Git vs Rsync Mental Model

Use Git for code and history. Use rsync for generated data/results.

Git understands commits, branches, and conflict detection. Rsync only compares files and paths. It can overwrite a newer local file with a remote file if you ask it to pull that path. That is why the sweep pull command is intentionally narrow: it pulls generated sweep outputs only and avoids pulling sweep source code back from Rivanna.

Safe commands for this sweep:

```
./scripts/rsync_push_to_hpc.sh sweep
./scripts/rsync_pull_from_hpc.sh sweep
```

Avoid for this sweep:

```
./scripts/rsync_push_to_hpc.sh all
./scripts/rsync_pull_from_hpc.sh all
```

Can I Delete The Old Local Results File?

Yes. The old local file:

```
sports/outputs/simulation_sweeps/faithful_537_sweep_results.jsonl
```

came from the aborted local run. It is safe to delete if you are switching to the Rivanna run.

The Rivanna run writes new outputs under:

```
sports/outputs/simulation_sweeps/rivanna_faithful_537/
```

Common Problems

`stage1_results.csv` missing

Stage 2 needs Stage 1 output. Run Stage 1 first, or use the dependency chain above.

`sports_net` not found

The Slurm scripts default to:

```
ENV_NAME="${ENV_NAME:-sports_net}"
```

If your Rivanna env has a different name:

```
ENV_NAME=my_env_name sbatch \
  sports/outputs/simulation_sweeps/rivanna_stage1_faithful_537.slurm
```

Do the same for Stage 2 and Merge.

I changed the number of shards

If you edit Stage 2 from 64 shards to another number, make the Slurm array and `N_SHARDS` match.

For example, 128 shards:

```
#SBATCH --array=0-127
N_SHARDS=128 sbatch \
  sports/outputs/simulation_sweeps/rivanna_stage2_array_faithful_537.slurm
N_SHARDS=128 sbatch \
  sports/outputs/simulation_sweeps/rivanna_merge_faithful_537.slurm
```